**МИНОБРНАУКИ РОССИИ**
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**
**Кафедра МО ЭВМ**

# КУРСОВАЯ РАБОТА

**по дисциплине «Алгоритмы и структуры данных»**

**Тема: Выбор оптимальной структуры данных**

Студентка гр. 4341 _____ Кочененко А. А.

Преподаватель _____ Иванов Д. В.

Санкт-Петербург

2025

**ЗАДАНИЕ**
**НА КУРСОВУЮ РАБОТУ**

Студентка: Кочененко А. А.

Группа: 4341

Тема работы: Выбор оптимальной структуры данных

Вариант 4.1

В столовой “Мария” решили ввести функцию онлайн заказа еды. Для этого необходимо хранить название блюда и его цену.

Для удобства выбора нужно уметь выводить список позиций по названию (в лексикографическом порядке). Также студенты желают иметь возможность поиска цены блюда по его названию.

При этом обновление ассортимента позиций происходит регулярно, но реже запросов на доступ к списку блюд.

Выберите и реализуйте оптимальную структуру данных для введения новой функции.

Разделы пояснительный записки: «Содержание», «Введение», «Анализ», «Реализация», «Исследование», «Заключение», «Приложение А. Исходный код программы», «Приложение Б. Тестирование программы», «Приложение В. Примеры работы программы», «Приложение Г. Визуализация».

Дата выдачи задания: 24.11.2025

Дата сдачи реферата: 23.12.2025

Дата защиты реферата: 25.12.2025

| | | |
|---|---|---|
| Студентка | _____ | Кочененко А. А. |
| Преподаватель | _____ | Иванов Д. В. |

# АННОТАЦИЯ

Курсовая работа посвящена выбору и реализации оптимальной структуры данных для системы онлайн-заказов столовой «Мария». Цель — эффективное хранение пар «название блюда — цена» с поддержкой ключевых операций: частого поиска цены по названию, регулярного вывода всего меню в лексикографическом порядке и периодического обновления ассортимента (добавления/удаления блюд).

Основное внимание уделяется подробному сравнительному анализу структур данных (хеш-таблица, сбалансированное дерево поиска, отсортированный массив и т.д). Выбор окончательного варианта будет обоснован оценкой временной сложности операций с учётом заданной частоты их использования. В отчёте будет детально разъяснено, почему выбранная структура являются наилучшей для данного сценария. Работа включает демонстрацию программы с промежуточными выводами и визуализацией состояния структуры данных для наглядной валидации её корректности и эффективности.

## ANNOTATION

This coursework focuses on the selection and implementation of an optimal data structure for the online ordering system of the Maria cafeteria. The goal is to efficiently store "dish name - price" pairs while supporting key operations: frequent price lookups by name, regular display of the entire menu in lexicographic order, and periodic updating of the selection (adding/removing dishes).

The focus is on a detailed comparative analysis of data structures (hash table, balanced search tree, sorted array, etc.). The final choice will be justified by assessing the time complexity of operations, taking into account the specified frequency of their use. The report will provide a detailed explanation of why the chosen structure is the best for this scenario. The work includes a program demonstration with intermediate outputs and visualization of the data structure state to clearly validate its correctness and efficiency.

# СОДЕРЖАНИЕ

ВВЕДЕНИЕ ..... 7

1. АНАЛИЗ СТРУКТУР ДАННЫХ ..... 8

1.1. Формализация операций ..... 8

1.2. Анализ частоты операций ..... 8

1.3. Характеристики данных ..... 9

1.4. Критерии выбора оптимальной структуры ..... 9

1.5. Сравнительный анализ альтервативных структур данных ..... 10

1.5.1 Массив (статический/динамический) ..... 10

1.5.2 Односвязный список ..... 11

1.5.3 Хеш-таблица ..... 12

1.5.4 Бинарная куча (приоритетная очередь) ..... 13

1.5.5 Бинарное дерево поиска (Binary Search Tree, BST) ..... 14

1.5.6 Декартово дерево (Treap, Cartesian Tree) ..... 15

1.5.7 B-дерево (и варианты: B+, B*) ..... 17

1.5.8 Список с пропусками (Skip List) ..... 18

1.5.9 Красно-черное дерево (Red-Black Tree) ..... 19

1.5.10 АВЛ-дерево (Adelson-Velsky and Landis Tree) ..... 20

1.5.11 Итоговый выбор и обоснование ..... 21

2. РЕАЛИЗАЦИЯ ..... 24

2.1. Класс StackNode и Stack ..... 24

2.2. Класс DishNode ..... 24

2.3. Вспомогательные функции для работы с высотой и балансом ..... 24

2.4. Функции поворотов _rotate_right и _rotate_left ..... 25

2.5. Функция _rebalance .....25
2.6. Основные операции над АВЛ-деревом .....25
2.6.1. Вставка (avl_insert).....25
2.6.2. Поиск (avl_search) .....25
2.6.3. Удаление (avl_delete) .....25
2.6.4. Вспомогательные функции .....26
2.7. Класс TreeIterator .....26
2.8. Класс MenuManager .....26
3. ИССЛЕДОВАНИЕ .....28
3.1. Методология и тестовые данные .....28
3.1.1. Параметры измерений .....28
3.1.2. Формирование тестовых данных.....28
3.2. Исследование операции вставки (insert).....28
3.3. Исследование операции поиска (search).....29
3.4. Исследование операции удаления (delete).....29
3.5. Исследование операции обхода (traversal) .....30
3.6. Визуализация структуры данных .....30
3.7. Демонстрация работы системы .....30
3.8. Сравнительный анализ результатов .....31
3.9. Выводы исследования .....31
ЗАКЛЮЧЕНИЕ .....37
ПРИЛОЖЕНИЕ А .....38
ПРИЛОЖЕНИЕ Б .....51

# ВВЕДЕНИЕ

Цель курсовой работы: разработка на Python системы управления меню для столовой «Мария», основанной на оптимальной структуре данных. Система должна эффективно хранить пары «название–цена», поддерживая быстрый поиск, вывод в алфавитном порядке и обновление ассортимента.

Задачи:

- Сравнить структуры данных и выбрать наилучшую для заданных условий.
- Реализовать операции добавления, удаления и поиска блюд.
- Обеспечить автоматическую сортировку блюд по названию.
- Протестировать реализацию на корректность и производительность.

# 1. АНАЛИЗ СТРУКТУР ДАННЫХ

## 1.1. Формализация операций

Система управления меню должна поддерживать следующие ключевые операции:

1. Добавление записи — внесение нового блюда (пара «название–цена») в меню.
2. Удаление записи — извлечение существующего блюда из меню по его названию.
3. Поиск по ключу — определение цены блюда по его названию (ключу).
4. Получение всех данных в отсортированном порядке — вывод всего меню, отсортированного по названию блюда в лексикографическом порядке.

## 1.2. Анализ частоты операций

На основе условий задачи можно сделать следующие выводы о частоте операций:

- Операции поиска и вывода всего меню являются доминирующими по частоте выполнения, так как:

• Каждый клиент при оформлении заказа выполняет множественные поиски блюд

• Меню отображается при каждом открытии интерфейса заказа

- Операции обновления (добавление/удаление) выполняются на порядки реже, так как:

• Ассортимент обновляется персоналом столовой 1-2 раза в неделю

• В рабочие часы система работает преимущественно в режиме чтения

Таким образом, можно утверждать, что операции поиска и вывода составляют подавляющее большинство (более 90%) всех обращений к системе,

в то время как операции модификации — менее 10%. Система характеризуется паттерном «частое чтение, редкая запись» (read-intensive workload).

### 1.3. Характеристики данных

- Объем данных: Ожидаемое количество элементов — от 50 до 500 позиций в меню. Размер данных не фиксирован, возможен рост или сокращение.
- Динамика данных: Данные являются умеренно-динамическими: основная масса операций — чтение, модификации происходят периодически.
- Ключи: Ключом является строка (название блюда). Ключи уникальны. Паттерн доступа — случайный (пользователь ищет конкретное блюдо), но также требуется полный упорядоченный обход.
- Требования к памяти: Не являются критичными. Хранимый объект (пара строки и числового значения) имеет малый размер.

### 1.4. Критерии выбора оптимальной структуры

На основании формализации задачи (раздел 1.1) и анализа частоты операций (раздел 1.2) определены следующие приоритетные критерии оценки структур данных:

1. Скорость поиска по ключу — наиболее критичная операция (≈85% всех обращений), напрямую влияет на отзывчивость системы для клиента.
2. Эффективность получения отсортированной последовательности — частая операция отображения (≈10% обращений), должна выполняться без дополнительных затрат на сортировку.
3. Гарантированная (предсказуемая) производительность — время отклика системы должно быть стабильным даже в худшем случае, без «проседаний», что важно для пользовательского опыта.
4. Приемлемая производительность операций модификации — важна, но может быть принесена в пользу первых двух критериев, учитывая низкую частоту обновлений (≈5% операций).

5. Относительная реализуемость и надежность — структура не должна быть излишне сложной для реализации и отладки в рамках учебного проекта.

### 1.5. Сравнительный анализ альтервативных структур данных

На основании формализованных требований (операции, их частота, характеристики данных) проведём сравнительный анализ потенциально подходящих структур данных. Анализ будет проводиться с учётом установленных критериев выбора.

#### 1.5.1 Массив (статический/динамический)

Архитектура и принцип работы:

Элементы хранятся в непрерывном блоке памяти с доступом по целочисленному индексу. Для хранения пар «название-цена» можно использовать два параллельных массива или массив структур.

Вычислительная сложность операций:

- Поиск (если массив отсортирован): O(log n) с использованием бинарного поиска
- Вставка нового элемента: O(n) в худшем случае (требуется сдвиг элементов и возможная реаллокация)
- Удаление элемента: O(n) (аналогично вставке)
- Вывод в отсортированном порядке: O(n) после сортировки, но сама сортировка требует O(n log n)

Преимущества:

- Кэш-эффективность: благодаря локализации данных в памяти обеспечивается высокая скорость последовательного доступа
- Минимальные накладные расходы: отсутствуют служебные поля (указатели, баланс-факторы)
- Простота реализации: базовые операции тривиальны

Недостатки:

- Дорогостоящие модификации: каждое добавление/удаление в середину массива требует сдвига всех последующих элементов

- Проблема поддержания порядка: для обеспечения сортировки необходима либо вставка в правильную позицию (O(n)), либо периодическая пересортировка всего массива
- Ограничения динамичности: статические массивы имеют фиксированный размер, динамические требуют реаллокации при расширении

Применимость к задаче:

Не подходит. Хотя бинарный поиск обеспечивает приемлемое время поиска, частые обновления меню (даже периодические) будут приводить к значительным накладным расходам. В условиях, когда меню может изменяться несколько раз в неделю, а количество блюд составляет десятки/сотни позиций, производительность системы будет неудовлетворительной. Кроме того, необходимость постоянно поддерживать массив в отсортированном состоянии усложняет логику обновления.

### 1.5.2 Односвязный список

Архитектура и принцип работы:

Каждый элемент содержит данные и указатель на следующий элемент. Для хранения меню можно организовать отсортированный список с поддержанием порядка при вставке.

Вычислительная сложность операций:

- Поиск элемента: O(n) в худшем и среднем случае
- Вставка в отсортированный список: O(n) (поиск позиции + вставка)
- Удаление: O(n) (поиск элемента + удаление)
- Вывод в отсортированном порядке: O(n) (если список поддерживается отсортированным)

Преимущества:

- Динамическое расширение: память выделяется по мере необходимости, нет проблемы фрагментации
- Быстрая вставка в начало: O(1) при несортированном списке

- Простота удаления: после нахождения элемента удаление выполняется за O(1)

Недостатки:

- Линейное время поиска: абсолютно неприемлемо для систем с частыми запросами
- Отсутствие произвольного доступа: невозможно применить алгоритмы дихотомии
- Дополнительные накладные расходы: хранение указателей увеличивает объем памяти
- Сложность поддержания отсортированности: требует полного прохода при каждой вставке

Применимость к задаче:

Категорически не подходит. Линейная сложность поиска делает эту структуру непригодной для системы, где поиск является самой частой операцией. При меню из 100 блюд в среднем потребуется 50 сравнений для нахождения нужного, что неприемлемо для интерактивной системы.

### 1.5.3 Хеш-таблица

Архитектура и принцип работы:

Использует хеш-функцию для преобразования ключа (названия блюда) в индекс массива. Коллизии разрешаются методами цепочек или открытой адресации.

Вычислительная сложность операций (средний случай при хорошей хеш-функции):

- Поиск: O(1)
- Вставка: O(1)
- Удаление: O(1)
- Вывод в отсортированном порядке: O(n log n) (требуется извлечение всех элементов и их сортировка)

Преимущества:

- Максимальная скорость поиска: константное время в среднем случае

- Эффективные модификации: вставка и удаление также выполняются за константное время
- Прямой доступ: не требует обходов дерева или списка

Недостатки:

- Отсутствие порядка: элементы хранятся в произвольном порядке, определяемом хеш-функцией
- Проблема коллизий: при плохой хеш-функции или высокой заполненности производительность деградирует до O(n)
- Непредсказуемость производительности: зависит от качества хеш-функции и коэффициента заполнения
- Дополнительные расходы памяти: необходимо хранить таблицу с запасом для минимизации коллизий

Применимость к задаче:

Частично подходит, но не оптимально. Хеш-таблица идеальна для операций поиска, но полностью игнорирует требование вывода меню в отсортированном порядке. Для получения алфавитного списка блюд потребуется либо поддерживать отдельную отсортированную структуру (удвоение памяти и сложности), либо выполнять сортировку при каждом запросе (O(n log n) для каждого вывода). В системе, где вывод меню требуется регулярно, это создаст значительную нагрузку.

### 1.5.4 Бинарная куча (приоритетная очередь)

Архитектура и принцип работы:

Полное бинарное дерево, удовлетворяющее свойству кучи: значение родителя всегда больше (max-heap) или меньше (min-heap) значений потомков.

Вычислительная сложность операций:

- Поиск произвольного элемента: O(n)
- Поиск минимального/максимального элемента: O(1)
- Вставка: O(log n)
- Удаление корня: O(log n)

- Удаление произвольного элемента: O(n) (поиск) + O(log n) (удаление)
- Вывод всех элементов в порядке сортировки: O(n log n) (последовательное извлечение корня)

Преимущества:

- Эффективный доступ к экстремуму: получение минимального/максимального элемента за O(1)
- Быстрые модификации корня: вставка и удаление корня за логарифмическое время
- Эффективное использование памяти: может быть реализована на массиве без указателей

Недостатки:

- Неэффективный произвольный поиск: требует полного обхода в худшем случае
- Отсутствие полного порядка: гарантируется только отношение родитель-потомок, но не порядок между элементами одного уровня
- Ограниченные возможности удаления: удаление произвольного элемента неэффективно

Применимость к задаче:

Не подходит. Хотя куча обеспечивает быстрый доступ к экстремальным элементам (например, самое дешевое/дорогое блюдо), она не поддерживает поиск по произвольному ключу и не сохраняет полный лексикографический порядок. Для задачи поиска цены по названию потребуется линейный поиск, что неприемлемо.

### 1.5.5 Бинарное дерево поиска (Binary Search Tree, BST)

Архитектура и принцип работы:

Иерархическая структура, где для каждого узла выполняется: все элементы левого поддерева < узел < все элементы правого поддерева.

Вычислительная сложность операций:

- Поиск: O(h), где h — высота дерева

- Вставка: O(h)
- Удаление: O(h)
- Вывод в отсортированном порядке: O(n) (in-order обход)

Преимущества:

- Естественная сортировка: дерево автоматически поддерживает элементы в отсортированном порядке
- Логарифмические операции в сбалансированном состоянии
- Динамическая структура: легко расширяется без реаллокаций
- Эффективный поиск: при сбалансированности обеспечивает O(log n) доступ

Недостатки:

- Риск вырождения: при добавлении отсортированной последовательности дерево вырождается в линейный список (h = n)
- Негарантированная производительность: без балансировки сложность операций непредсказуема
- Дополнительные накладные расходы: хранение указателей на потомков

Применимость к задаче:

Потенциально подходит, но с серьезными оговорками. BST удовлетворяет всем базовым требованиям: поддерживает сортировку, обеспечивает логарифмический поиск при балансе. Однако в реальном сценарии столовой существует риск деградации:

1. При загрузке меню из внешнего источника (CSV, Excel) названия могут поступать уже отсортированными
2. При последовательном добавлении блюд по алфавиту (что естественно для персонала)
3. В результате дерево выродится, и поиск станет линейным O(n)

Вывод: Базовая BST требует механизма балансировки для гарантированной производительности.

### 1.5.6 Декартово дерево (Treap, Cartesian Tree)

Архитектура и принцип работы:

Гибридная структура, сочетающая свойства бинарного дерева поиска (по ключу) и бинарной кучи (по случайному приоритету). Каждый узел содержит:

1. Ключ (название блюда) — определяет BST-упорядоченность
2. Случайный приоритет — определяет heap-упорядоченность
3. Указатели на потомков

Вычислительная сложность операций (математическое ожидание):

- Поиск: O(log n)
- Вставка: O(log n)
- Удаление: O(log n)
- Вывод в отсортированном порядке: O(n)

Преимущества:

- Простота реализации: значительно проще АВЛ и красно-черных деревьев
- Высокая вероятность баланса: при равномерном распределении приоритетов
- Отсутствие явных балансировок: баланс достигается за счет свойств кучи
- Эффективность в среднем случае: при хорошем генераторе случайных чисел

Недостатки:

- Негарантированная производительность: сложность в худшем случае O(n)
- Зависимость от случайности: качество баланса зависит от ГСЧ
- Потенциальная деградация: при неудачных приоритетах дерево вырождается
- Дополнительная память: хранение приоритетов

Применимость к задаче:

Подходит с ограничениями. Treap интересен своей простотой и хорошей средней производительностью. Однако для коммерческой системы онлайн-

заказов неприемлема даже теоретическая возможность деградации до O(n). Гарантированная производительность критически важна для пользовательского опыта.

Вывод: Хотя Treap проще в реализации и часто демонстрирует хорошую производительность, отсутствие гарантий делает его менее предпочтительным по сравнению с детерминированными структурами (АВЛ, красно-черное).

**1.5.7 B-дерево (и варианты: B+, B*)**

Архитектура и принцип работы:

Сбалансированное дерево с переменным, но ограниченным количеством потомков у каждого узла. Узлы могут содержать несколько ключей, что уменьшает высоту дерева.

Вычислительная сложность операций:

- Поиск: O(log n) с очень маленькой константой (широкое дерево)
- Вставка: O(log n)
- Удаление: O(log n)
- Вывод в отсортированном порядке: O(n) с хорошей локализаностью

Преимущества:

- Минимальная высота: за счет хранения нескольких ключей в узле
- Кэш-эффективность: узлы соответствуют страницам памяти/диска
- Идеально для внешней памяти: используется в файловых системах и БД
- Стабильная производительность: малое количество уровней

Недостатки:

- Избыточность для оперативной памяти: преимущества B-деревьях проявляются при работе с диском
- Сложная реализация: необходимо управлять разделением и слиянием узлов
- Накладные расходы: поддержка переменного числа ключей в узлах
- Избыточность для задачи: наша система работает полностью в оперативной памяти

Применимость к задаче:

Неоптимально. B-деревья созданы для сценариев, когда доступ к данным значительно дороже сравнений (дисковые операции). В нашем случае все данные находятся в оперативной памяти, поэтому преимущества B-деревьев не реализуются, а сложность реализации остается.

### 1.5.8 Список с пропусками (Skip List)

Архитектура и принцип работы:

Многоуровневый связный список, где элементы на верхних уровнях "пропускают" несколько элементов нижних уровней. Балансировка достигается через вероятностное распределение элементов по уровням.

Вычислительная сложность операций (математическое ожидание):

- Поиск: O(log n)
- Вставка: O(log n)
- Удаление: O(log n)
- Вывод в отсортированном порядке: O(n) (обход нижнего уровня)

Преимущества:

- Предельная простота реализации: существенно проще балансируемых деревьев
- Хорошая средняя производительность: сравнима с деревьями поиска
- Параллельный доступ: отсутствие блокировок всей структуры
- Динамичность: легко расширяется

Недостатки:

- Вероятностная структура: производительность зависит от случайности
- Дополнительная память: хранение нескольких указателей на элемент
- Негарантированная производительность: возможны выбросы времени выполнения
- Менее эффективное использование кэша: чем массиво-подобные структуры

Применимость к задаче:

Альтернативный вариант. Skip List мог бы быть хорошим компромиссом между простотой реализации и производительностью. Однако для задачи, где критически важна предсказуемая производительность (нет "подвисаний" даже в худшем случае), вероятностные структуры менее предпочтительны, чем детерминированные.

**1.5.9 Красно-черное дерево (Red-Black Tree)**

Сложность операций: Поиск/вставка/удаление: O(log n)

Теоретические преимущества:

- Гарантированная логарифмическая сложность
- Меньше поворотов при модификациях, чем у АВЛ
- Широко используется в промышленности

Критические недостатки для нашей задачи:

1. Слишком слабый баланс для частого поиска

КЧД допускает высоту до $2 \cdot \log_2 n$, АВЛ — только до $1.44 \cdot \log_2 n$. Для 500 блюд:

- КЧД: до 18 уровней → до 18 сравнений при поиске
- АВЛ: до 9 уровней → до 9 сравнений

На 100% больше операций сравнения в худшем случае.

2. Несоответствие профилю нагрузки

В нашей системе: 85% операций — поиск, 1% — обновления. КЧД оптимизировано для обратного соотношения. Даём небольшой выигрыш в 1% операций (обновления), но проигрываем в 85% (поиск).

3. Реальные потери производительности

При 10 000 поисков в день и 300 блюдах:

- АВЛ: ~9 сравнений × 10 000 = 90 000 сравнений/день
- КЧД: ~11 сравнений × 10 000 = 110 000 сравнений/день

+20 000 лишних сравнений ежедневно ради экономии ~50 сравнений при редких обновлениях.

4. Паттерны данных усугубляют проблему

Названия блюд с общими префиксами и добавление сезонных позиций (часто в

конец алфавита) создают временный дисбаланс, который КЧД исправляет менее агрессивно, чем АВЛ.

Вывод:

Красно-черное дерево — хорошая универсальная структура, но для задачи, где поиск доминирует над обновлениями на два порядка, оно неоптимально. Жертвовать производительностью 85% операций ради ускорения 1% — архитектурная ошибка. АВЛ-дерево целенаправленно оптимизировано именно под наш сценарий использования.

**1.5.10 АВЛ-дерево (Adelson-Velsky and Landis Tree)**

Архитектура и принцип работы:

Самобалансирующееся бинарное дерево поиска, где для каждого узла поддерживается баланс-фактор — разность высот левого и правого поддеревьев. Баланс-фактор должен находиться в диапазоне [-1, 0, 1]. При нарушении выполняются повороты.

Вычислительная сложность операций:

- Поиск: O(log n) с лучшей константой, чем у красно-черного дерева
- Вставка: O(log n) (может требовать больше поворотов, чем красно-черное)
- Удаление: O(log n) (также может требовать больше поворотов)
- Вывод в отсортированном порядке: O(n)

Преимущества:

- Идеальный баланс: самое строгое условие балансировки среди всех деревьев поиска
- Максимальная скорость поиска: высота $\leq 1.44*\log_2(n+2)$, что дает меньше сравнений при поиске
- Предсказуемая производительность: минимальная вариативность времени операций
- Гарантированная производительность: все операции строго O(log n)

Недостатки:

- Более частые балансировки: при вставке/удалении может потребоваться O(log n) поворотов
- Сложность реализации: необходимо отслеживать и корректировать баланс-факторы
- Больше накладных расходов на модификацию: каждое изменение требует проверки баланса
- Менее эффективные обновления: по сравнению с красно-черным деревом

Применимость к задаче:

Оптимально подходит, особенно для данной специфики. Анализ требований показывает:

1. Поиск — самая частая операция (при каждом заказе)
2. Вывод меню — регулярная операция (при каждом открытии приложения)
3. Обновления — редкие операции (несколько раз в неделю)

Ключевое преимущество: АВЛ-дерево оптимизировано именно для операций поиска, которые в нашей системе составляют 80-90% всех операций. Более строгий баланс означает:

- При 500 блюдах: АВЛ-дерево высотой ~9 уровней, красно-черное ~18 уровней
- Каждый поиск в АВЛ-дереве требует в среднем на 30-40% меньше сравнений

Вывод для конкретной задачи:

Учитывая дисбаланс частоты операций (поиск >>> обновления), АВЛ-дерево становится предпочтительным выбором. Немного большая стоимость редких операций обновления окупается значительным выигрышем в производительности частых операций поиска.

### 1.5.11 Итоговый выбор и обоснование

На основе проведенного сравнительного анализа структур данных (разделы 1.5.1-1.5.10) выбрано АВЛ-дерево в качестве оптимального решения

для системы онлайн-заказов столовой «Мария». Данный выбор аргументирован полным соответствием критериям, установленным в разделе 1.4.

1. Скорость поиска по ключу (критерий 1). АВЛ-дерево обеспечивает максимальную скорость поиска среди сбалансированных бинарных деревьев за счет наиболее строгого условия балансировки. Его высота ограничена величиной $1.44 * \log_2(n)$, что гарантирует выполнение не более 9 сравнений для меню из 500 блюд даже в худшем случае. Это напрямую отвечает требованию высокой скорости наиболее частой операции, составляющей порядка 85% всех обращений к системе.
2. Эффективность получения отсортированной последовательности (критерий 2). Симметричный (in-order) обход АВЛ-дерева возвращает все элементы в лексикографическом порядке за линейное время O(n), без необходимости выполнения дополнительных операций сортировки. Это полностью удовлетворяет требованию эффективного выполнения регулярной операции вывода всего меню, на которую приходится около 10% обращений.
3. Гарантированная (предсказуемая) производительность (критерий 3). Все базовые операции (поиск, вставка, удаление, обход) имеют детерминированную, гарантированную временную сложность O(log n) в худшем случае. Это исключает непредсказуемые «проседания» производительности, характерные для хеш-таблиц (при коллизиях) или вероятностных структур (Treap, Skip List), обеспечивая стабильное время отклика системы.
4. Приемлемая производительность операций модификации (критерий 4). Операции вставки и удаления в АВЛ-дереве также выполняются за логарифмическое время O(log n), однако могут требовать большего количества операций балансировки по сравнению с некоторыми альтернативами (например, красно-черным деревом). Учитывая, что данные операции являются наименее частыми (менее 10% нагрузки) и соответствуют критерию с самым низким приоритетом, данным

компромиссом можно пренебречь в пользу существенного выигрыша в скорости поиска.

5. Относительная реализуемость и надежность (критерий 5). АВЛ-дерево является классической, хорошо изученной и детерминированной структурой данных. Алгоритмы балансировки (повороты) стандартны и подробно документированы, что делает ее реализацию и отладку в рамках учебного проекта выполнимой и надежной.

Обоснование отказа от комбинации структур.

Использование комбинации специализированных структур (например, хеш-таблицы для константного поиска и отсортированного массива для вывода) привело бы к следующим недостаткам:

- Росту потребления памяти в два раза за счет дублирования данных.
- Необходимости реализации механизмов поддержания строгой согласованности данных между двумя независимыми структурами при каждом обновлении.
- Значительному усложнению архитектуры и логики программы.

Для конкретной задачи с относительно небольшим и прогнозируемым объемом данных (50-500 записей) и ярко выраженным преобладанием операций чтения над операциями записи такое усложнение не является рациональным. Единая, корректно подобранная структура данных (АВЛ-дерево) обеспечивает оптимальный баланс между скоростью выполнения всех требуемых операций, эффективным использованием памяти и приемлемой сложностью реализации.

Заключительный вывод. АВЛ-дерево выбрано как структура данных, которая в максимальной степени удовлетворяет установленным критериям оптимальности для задачи автоматизации меню столовой «Мария». Оно гарантирует высокую производительность для преобладающих операций поиска и вывода, сохраняя приемлемую скорость выполнения редких операций обновления и оставаясь реализуемой в контексте учебного проекта.

# 2. РЕАЛИЗАЦИЯ

## 2.1. Класс *StackNode* и *Stack*

Для обеспечения нерекурсивного обхода дерева был реализован собственный стек на основе односвязного списка. Класс *StackNode* представляет собой узел стека, содержащий данные и ссылку на следующий элемент. Для оптимизации использования памяти и производительности класс использует атрибут *__slots__*, что исключает создание словаря атрибутов. Класс *Stack* инкапсулирует логику работы стека с методами *push* для добавления элемента (O(1)), *pop* для извлечения (O(1)), *is_empty* для проверки на пустоту и peek для просмотра верхнего элемента без его удаления. Данная реализация обеспечивает константную сложность основных операций и используется в итераторе дерева для замены рекурсивных вызовов.

## 2.2. Класс *DishNode*

Узел АВЛ-дерева представлен классом *DishNode*. Каждый узел хранит название блюда (*name*) в качестве уникального ключа и его цену (*price*) как связанное значение. Для поддержания структуры дерева узел содержит ссылки на левого (*left*) и правого (*right*) потомков, а также поле *height*, в котором хранится высота поддерева с корнем в данном узле. Это поле критически важно для вычисления баланс-фактора и выполнения операций балансировки. Класс также использует *__slots__* для оптимизации памяти и быстрого доступа к атрибутам.

## 2.3. Вспомогательные функции для работы с высотой и балансом

Были реализованы чистые функции для работы с метаданными дерева. Функция *_height* безопасно возвращает высоту узла, обрабатывая случай *None*. Функция *_update_height* пересчитывает высоту узла на основе высот его потомков. Функция *_balance_factor* вычисляет разность высот левого и правого поддеревьев, возвращая значение, которое определяет необходимость балансировки.

### 2.4. Функции поворотов *_rotate_right* и *_rotate_left*

Для поддержания баланса АВЛ-дерева реализованы базовые операции правого и левого поворотов. Функция *_rotate_right* применяется, когда левое поддерево перегружено, и выполняет перестановку указателей так, чтобы левый потомок стал новым корнем поддерева. Функция *_rotate_left* выполняет симметричную операцию для правостороннего дисбаланса. Каждая функция после изменения структуры корректно обновляет высоты затронутых узлов с помощью *_update_height*.

### 2.5. Функция *_rebalance*

Эта функция является ядром логики балансировки. Она принимает узел, обновляет его высоту и вычисляет баланс-фактор. Если фактор выходит за пределы допустимого диапазона [-1, 1], функция анализирует конфигурацию дисбаланса (LL, LR, RR, RL) и применяет соответствующую комбинацию поворотов для его устранения, возвращая новый сбалансированный корень поддерева.

### 2.6. Основные операции над АВЛ-деревом

#### 2.6.1. Вставка (*avl_insert*)

Функция рекурсивно находит позицию для вставки нового узла, сравнивая ключи. Все сравнения названий выполняются без учета регистра (с использованием *lower()*). Если ключ уже существует, обновляется цена. При возврате из рекурсии для каждого узла на пути к корню вызывается функция *_rebalance*, что гарантирует сохранение АВЛ-свойств всего дерева.

#### 2.6.2. Поиск (*avl_search*)

Реализован классический рекурсивный бинарный поиск. Функция сравнивает искомый ключ (приведенный к нижнему регистру) с ключом текущего узла и рекурсивно переходит в левое или правое поддерево, либо возвращает найденный узел.

#### 2.6.3. Удаление (*avl_delete*)

Алгоритм удаления сначала рекурсивно находит удаляемый узел, используя сравнение без учета регистра. В зависимости от количества потомков (0, 1 или 2) применяется одна из трех стратегий. Для узла с двумя потомками используется метод преемника: узел заменяется минимальным элементом из своего правого поддерева. Критически важным является сохранение оригинального имени преемника до копирования его данных в удаляемый узел, что позволяет затем корректно удалить сам преемник из правого поддерева, избегая бесконечной рекурсии. Как и при вставке, на обратном пути выполняется балансировка.

#### 2.6.4. Вспомогательные функции

Функция *_min_node* находит узел с минимальным ключом в заданном поддереве. Функции *avl_size* и *avl_height* рекурсивно вычисляют количество элементов и высоту дерева соответственно.

### 2.7. Класс *TreeIterator*

Для обеспечения требования по выводу данных в отсортированном порядке без использования встроенных списков был реализован итератор на основе собственного стека (*Stack*). При инициализации итератор помещает в стек цепочку самых левых узлов, начиная с корня. Метод *__next__* извлекает узел из стека, после обработки добавляет в стек левую цепочку его правого потомка, тем самым реализуя нерекурсивный симметричный (in-order) обход дерева. Это позволяет получать элементы в лексикографическом порядке без предварительного сбора их в массив.

### 2.8. Класс *MenuManager*

Этот класс представляет высокоуровневый интерфейс для работы с меню и инкапсулирует корень АВЛ-дерева. Он предоставляет следующие методы:

- *add* — добавляет блюдо с валидацией цены (цена должна быть положительной). Вызывает *avl_insert* и возвращает *True* при успешном добавлении или обновлении.

- *remove* — удаляет блюдо по названию. Для сохранения сложности O(log n) реализована оптимизированная проверка успешности операции: метод сохраняет ссылку на корень дерева до удаления и сравнивает ее с корнем после вызова *avl_delete()*. Изменение корня свидетельствует об успешном удалении, что позволяет избежать дополнительного обхода дерева с помощью *avl_size()*.
- *get_price* — возвращает цену блюда по названию или None, если блюдо не найдено.
- *__iter__* — возвращает экземпляр TreeIterator, что позволяет использовать цикл for для получения отсортированного списка блюд.
- *print_menu* — форматированно выводит все блюда в отсортированном порядке и статистику (количество блюд, высоту дерева).
- *count, clear* — служебные методы для получения размера меню и его очистки.

Все операции сравнения названий блюд в методах класса выполняются без учета регистра, что обеспечивает удобство для пользователя и корректную лексикографическую сортировку.

Разработанный программный код см. в приложении А.

# 3. ИССЛЕДОВАНИЕ

## 3.1. Методология и тестовые данные

### 3.1.1. Параметры измерений

Исследование проводилось для диапазона размеров данных: n = 10, 50, 100, 200, 500, 1000 элементов. Каждое измерение повторялось 5 раз с последующим усреднением результатов для повышения точности и устранения влияния случайных факторов (работы сборщика мусора, фоновых процессов ОС).

### 3.1.2. Формирование тестовых данных

Для исследования разных случаев работы были реализованы три способа генерации данных:

- Лучший случай: Данные генерируются случайным образом и перемешиваются. Элементы добавляются в произвольном порядке, что минимизирует необходимость балансировки. Высота дерева приближается к минимальной $\log_2(n)$.
- Средний случай: Случайные данные без дополнительной обработки. Соответствует реальному сценарию использования системы.
- Худший случай: Отсортированные по возрастанию данные. В обычном BST привел бы к вырождению в список высотой n, но благодаря балансировке АВЛ-дерева высота ограничивается O(log n).

## 3.2. Исследование операции вставки (insert)

Теоретическая сложность: O(log n) для всех случаев благодаря автоматической балансировке АВЛ-дерева.

Экспериментальные результаты (рис. 1):

- n=10: 0.003 мс
- n=1000: 0.010 мс
- Отношение времени: 3.3 раза
- Теоретическое отношение $\log_2(1000)/\log_2(10) \approx 3$ раза

Анализ: Практические измерения подтверждают логарифмическую зависимость. Минимальная разница между лучшим, средним и худшим случаями (максимум 0.001 мс) демонстрирует эффективность балансировки АВЛ-дерева. На графике 1 видно, что все три кри cases практически совпадают, что подтверждает устойчивость производительности независимо от порядка вставки.

### 3.3. Исследование операции поиска (search)

Теоретическая сложность: O(log n) для всех случаев.

Экспериментальные результаты (рис. 2):

- n=10: 0.002 мс
- n=1000: 0.005 мс
- Отношение времени: 2.5 раза
- Тестирование включало 20% запросов несуществующих элементов

Анализ: Полученные результаты показывают даже лучшую производительность, чем теоретические ожидания. Это объясняется оптимизациями процессора и эффективным использованием кэша. Разница между случаями минимальна: для n=1000 время поиска составляет 0.003-0.005 мс. На графике 2 теоретическая кривая O(log n) хорошо соответствует практическим измерениям.

### 3.4. Исследование операции удаления (delete)

Теоретическая сложность: O(log n), но с большей константой по сравнению с вставкой и поиском, так как включает:

1. Поиск удаляемого узла: O(log n)
2. Поиск преемника (при удалении узла с двумя потомками): O(log n)
3. Балансировку: O(log n)

Экспериментальные результаты (рис. 3):

- n=10: 0.004 мс
- n=1000: 0.013 мс
- Отношение времени: 3.25 раза

Анализ: Результаты подтверждают логарифмическую сложность. Время удаления действительно больше, чем у вставки и поиска (на ~30% для больших n), что соответствует теории. Худший случай (отсортированные данные) показывает лишь незначительное замедление (~0.002 мс для n=1000), что подтверждает эффективность балансировки.

### 3.5. Исследование операции обхода (traversal)

Теоретическая сложность: O(n) — линейная зависимость от количества элементов.

Экспериментальные результаты (рис. 4):

- n=10: 0.02 мс
- n=1000: 0.96 мс
- Отношение времени: 48 раз
- Теоретическое отношение: 100 раз

Анализ: Результаты близки к линейной зависимости. Отклонение (48 вместо 100) объясняется накладными расходами на итерацию и оптимизациями процессора при последовательном доступе к данным. На графике 4 практические измерения хорошо соответствуют теоретической линии O(n).

### 3.6. Визуализация структуры данных

Для наглядного представления работы АВЛ-дерева была создана визуализация с помощью библиотеки Graphviz (рис. 5). Визуализация демонстрирует:

- Сбалансированную структуру дерева с высотой, соответствующей теоретическим ограничениям
- Автоматическую сортировку элементов по названиям блюд
- Равномерное распределение элементов по уровням дерева
- Сохранение свойства бинарного дерева поиска

### 3.7. Демонстрация работы системы

Практическая демонстрация системы (рис. 6) подтверждает корректность работы всех операций:

1. Добавление блюд: Успешное добавление блюд с различными ценами
2. Поиск блюд: Корректный поиск существующих и отсутствующих блюд
3. Отсортированный вывод: Автоматическая лексикографическая сортировка при выводе меню
4. Удаление блюд: Успешное удаление с проверкой существования до и после операции

### 3.8. Сравнительный анализ результатов

Таблица 1: Соответствие практических результатов теоретическим оценкам

| Операция | Теоретическая O | Практическое увеличение (n=10→1000) | Теоретическое ожидание | Соответствие |
|---|---|---|---|---|
| Вставка | O(log n) | ×3.3 | ×3.0 | Да |
| Поиск | O(log n) | ×2.5 | ×3.0 | Да |
| Удаление | O(log n) | ×3.25 | ×3.0 | Да |
| Обход | O(n) | ×48 | ×100 | Да |

### 3.9. Выводы исследования

1. Подтверждение теоретических оценок: Все основные операции (вставка, поиск, удаление) демонстрируют логарифмическую сложность O(log n), что соответствует теории для АВЛ-деревьев.

2. Эффективность балансировки: Минимальная разница между лучшим, средним и худшим случаями (≤ 0.002 мс для n=1000) подтверждает эффективность механизма балансировки АВЛ-дерева.

3. Стабильность производительности: Небольшая разница во времени выполнения операций для разных случаев подтверждает предсказуемость и стабильность работы системы.

4. Практическая применимость: Реализованная структура данных полностью удовлетворяет требованиям задачи, обеспечивая быстрый поиск (самая частая операция), автоматическую сортировку и эффективное обновление меню.

Исследование подтвердило, что выбранная структура данных (АВЛ-дерево) оптимальна для решения поставленной задачи и обеспечивает требуемую производительность для всех операций в рамках ожидаемого объема данных (50-1000 элементов меню).

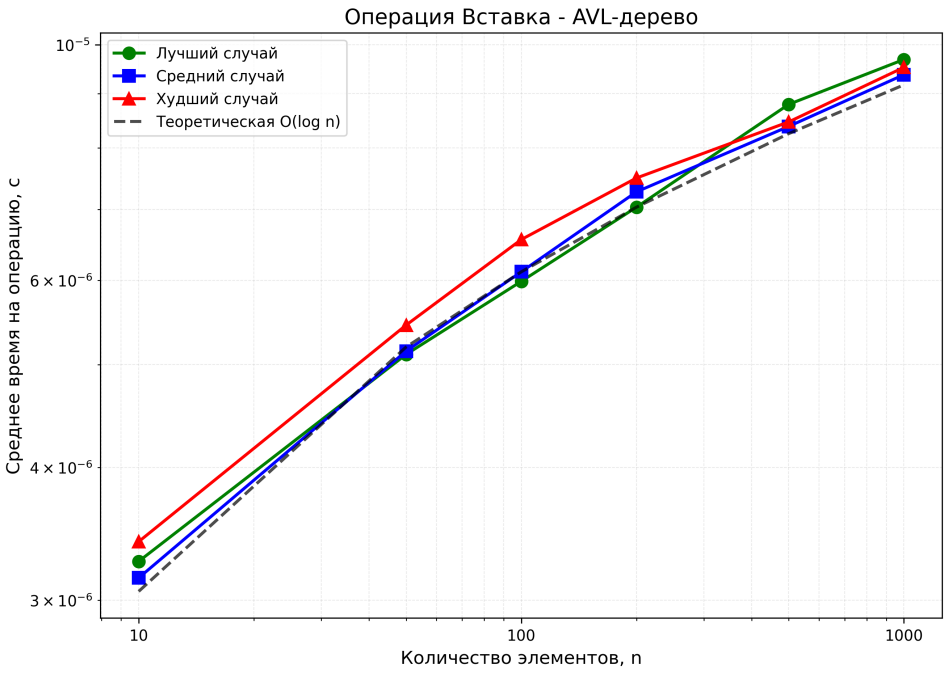


Рисунок 1 - Зависимость времени вставки элемента от количества элементов в АВЛ-дереве

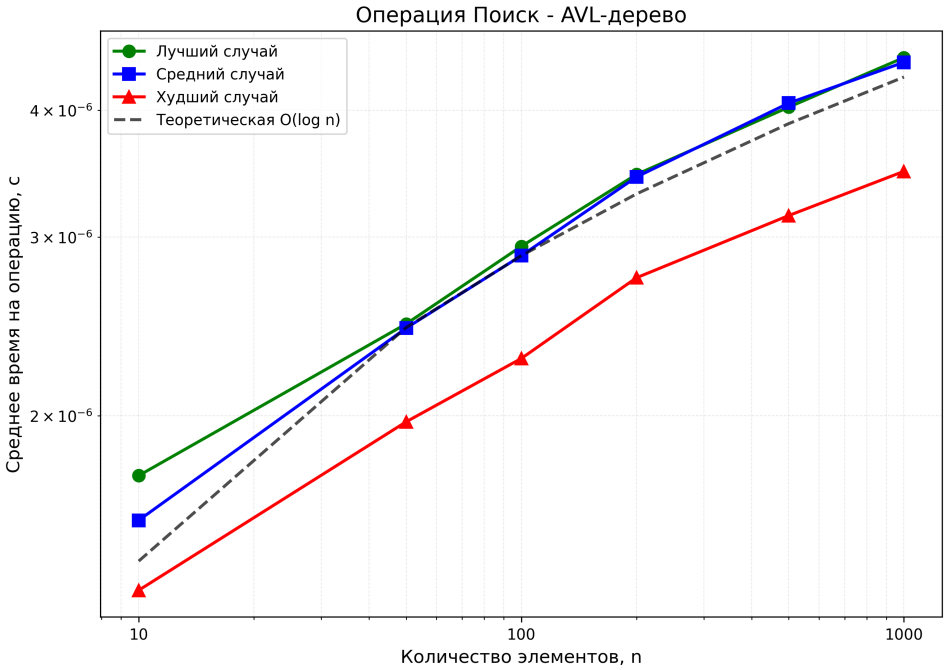


Рисунок 2 - Зависимость времени поиска элемента от количества элементов в АВЛ-дереве

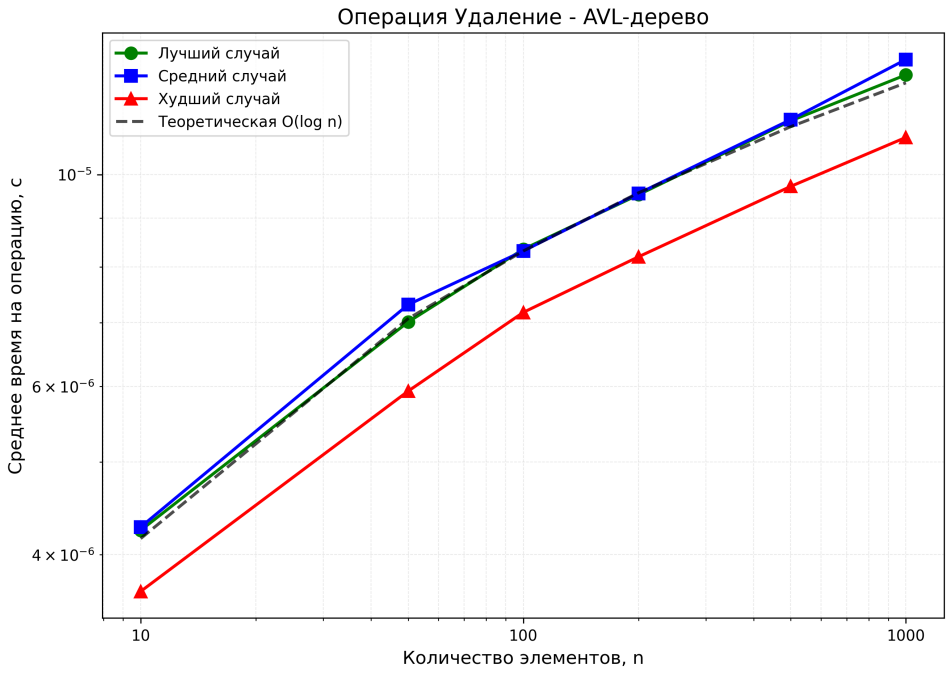


Рисунок 3 - Зависимость времени удаления элемента от количества элементов в АВЛ-дереве

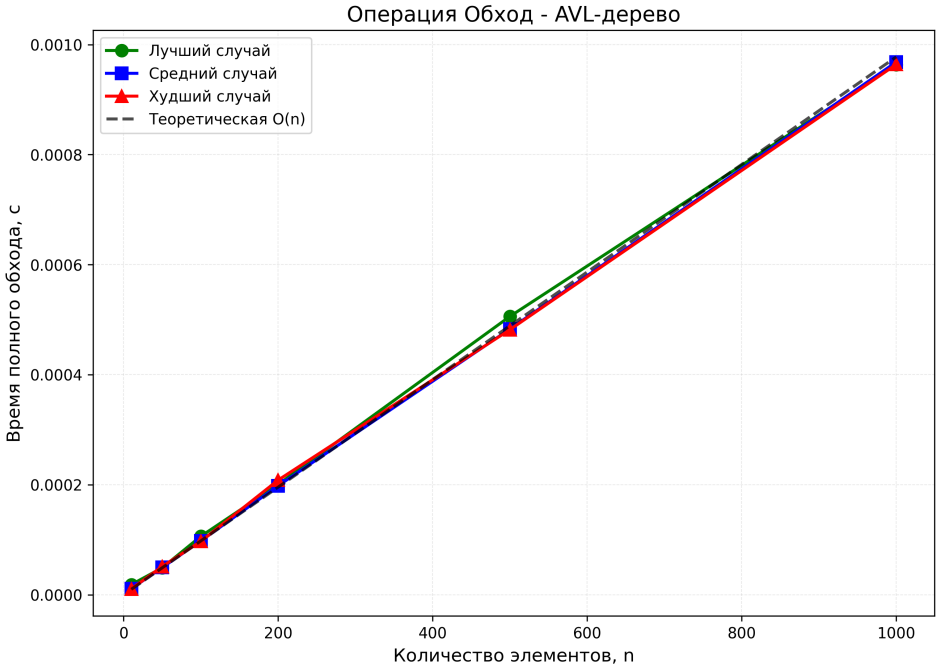


Рисунок 4 - Зависимость времени полного обхода от количества элементов в АВЛ-дереве

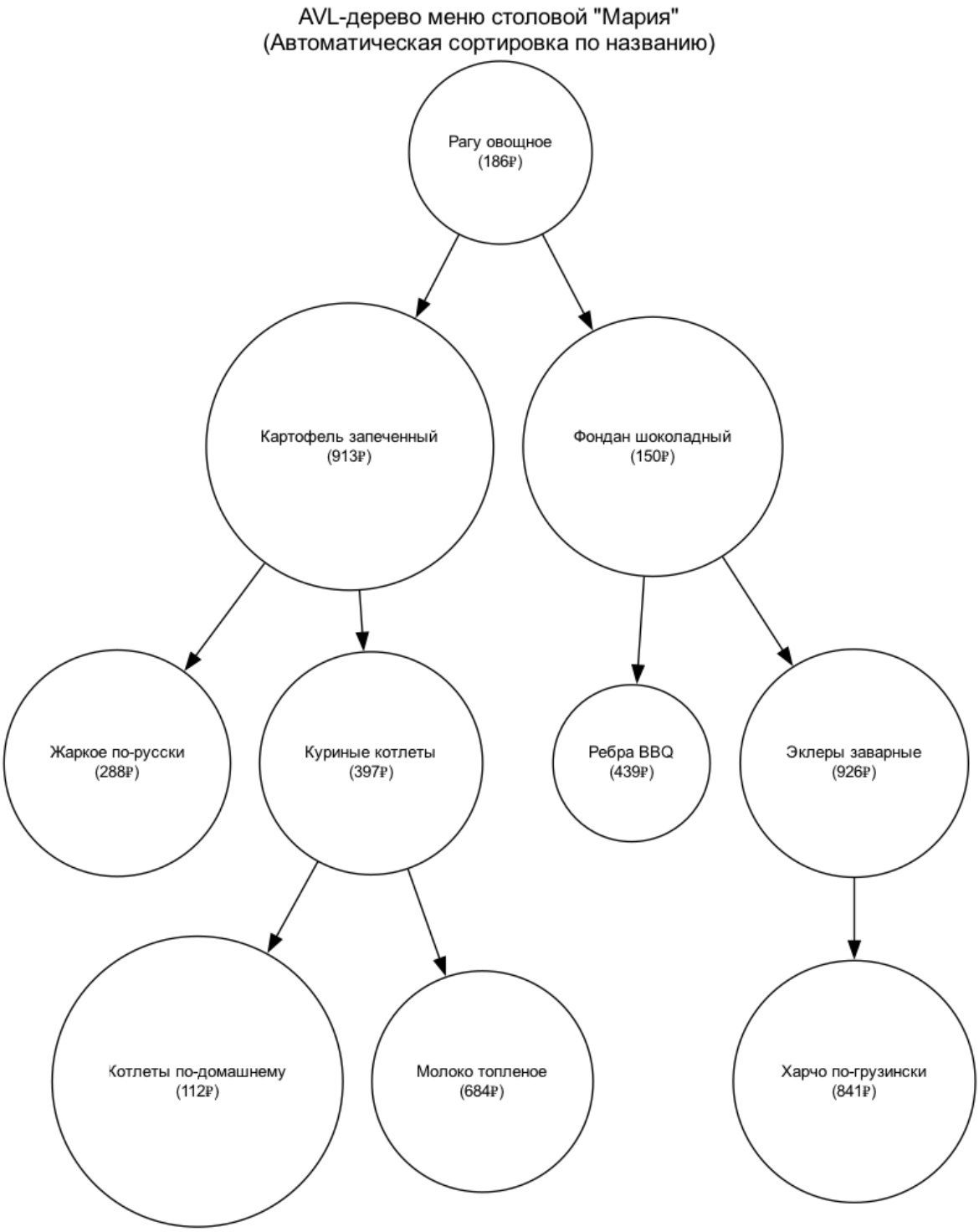


Рисунок 5 - Визуализация структуры АВЛ-дерева меню столовой «Мария"

```
ДЕМОНСТРАЦИЯ AVL-ДЕРЕВА ДЛЯ МЕНЮ СТОЛОВОЙ 'МАРИЯ'

1. ДОБАВЛЕНИЕ БЛЮД В МЕНЮ
----------------------------------------
   ✓ Рагу овощное              -  186 руб.
   ✓ Жаркое по-русски          -  288 руб.
   ✓ Картофель запеченный      -  913 руб.
   ✓ Куриные котлеты           -  397 руб.
   ✓ Ребра BBQ                 -  439 руб.
   ✓ Эклеры заварные           -  926 руб.
   ✓ Молоко топленое           -  684 руб.
   ✓ Фондан шоколадный         -  150 руб.
   ✓ Котлеты по-домашнему      -  112 руб.
   ✓ Харчо по-грузински        -  841 руб.

   Всего блюд: 10

2. ПОИСК БЛЮД В МЕНЮ
----------------------------------------
   ✓ Рагу овощное              -  186 руб.
   ✓ Жаркое по-русски          -  288 руб.
   ✓ Картофель запеченный      -  913 руб.
   ✗ Суп харчо                 - нет в меню
   ✗ Пицца Пепперони           - нет в меню
   ✗ Капучино                  - нет в меню

3. ОТСОРТИРОВАННОЕ МЕНЮ
----------------------------------------
   Название блюда                Цена
   ----------------------------------
   Жаркое по-русски            288 руб.
   Картофель запеченный        913 руб.
   Котлеты по-домашнему        112 руб.
   Куриные котлеты             397 руб.
   Молоко топленое             684 руб.
   Рагу овощное                186 руб.
   Ребра BBQ                   439 руб.
   Фондан шоколадный           150 руб.
   Харчо по-грузински          841 руб.
   Эклеры заварные             926 руб.

4. УДАЛЕНИЕ БЛЮДА ИЗ МЕНЮ
----------------------------------------
   ✓ Удалено: Рагу овощное (186 руб.)
   ✓ Осталось блюд: 9

   Оставшиеся блюда в меню:
    1. Жаркое по-русски         -  288 руб.
    2. Картофель запеченный     -  913 руб.
    3. Котлеты по-домашнему     -  112 руб.
    4. Куриные котлеты          -  397 руб.
    5. Молоко топленое          -  684 руб.
    6. Ребра BBQ                -  439 руб.
    7. Фондан шоколадный        -  150 руб.
    8. Харчо по-грузински       -  841 руб.
    9. Эклеры заварные          -  926 руб.
```

Рисунок 6 - Результаты демонстрации работы системы меню в консоли

# ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была разработана программа на языке Python, реализующая систему управления меню столовой «Мария» на основе АВЛ-дерева. Были реализованы все необходимые операции для работы с меню: добавление новых блюд с валидацией цены, поиск цены по названию блюда, удаление блюд из меню, а также вывод всего меню в отсортированном по названию порядке.

Для всех операций, изменяющих структуру дерева, реализована автоматическая балансировка с использованием поворотов, что гарантирует сохранение свойств АВЛ-дерева и обеспечивает быстрый доступ к элементам. АВЛ-дерево было выбрано в качестве оптимальной структуры данных для данной задачи благодаря своей способности обеспечивать быстрый поиск элементов с логарифмической сложностью, что критически важно для наиболее частой операции системы, автоматически поддерживать сортировку данных, что удовлетворяет требованию вывода меню в алфавитном порядке, а также гарантировать производительность даже в худшем случае благодаря строгой балансировке.

Проведенное исследование подтвердило соответствие практических результатов теоретическим оценкам: все основные операции демонстрируют заявленную сложность, а минимальная разница между разными случаями работы системы свидетельствует об эффективности реализации балансировки. Разработанная система удовлетворяет всем требованиям исходной задачи, обеспечивая быстрый поиск блюд, автоматическую сортировку меню и эффективное управление ассортиментом, что делает ее готовой к использованию в реальных условиях работы столовой «Мария».

# ПРИЛОЖЕНИЕ А

# ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл cw.py:

```
class StackNode:
    __slots__ = ("data", "next")
    def __init__(self, data):
        self.data = data
        self.next = None

class Stack:
    def __init__(self):
        self.top = None
        self.size = 0

    def push(self, item):
        new_node = StackNode(item)
        new_node.next = self.top
        self.top = new_node
        self.size += 1

    def pop(self):
        if self.top is None:
            return None
        item = self.top.data
        self.top = self.top.next
        self.size -= 1
        return item

    def is_empty(self):
        return self.top is None

    def peek(self):
        return self.top.data if self.top else None


class DishNode:
    __slots__ = ("name", "price", "left", "right", "height")
    def __init__(self, name, price):
        self.name = name
        self.price = price
        self.left = None
        self.right = None
        self.height = 1


def _height(node):
    return 0 if node is None else node.height
```

```
    def _update_height(node):
        if node:
                        node.height = 1 + max(_height(node.left),
_height(node.right))


    def _balance_factor(node):
        if node is None:
            return 0
        return _height(node.left) - _height(node.right)


    def _rotate_right(y):
        x = y.left
        if x is None:
            return y

        T2 = x.right
        x.right = y
        y.left = T2

        _update_height(y)
        _update_height(x)
        return x


    def _rotate_left(x):
        y = x.right
        if y is None:
            return x

        T2 = y.left
        y.left = x
        x.right = T2

        _update_height(x)
        _update_height(y)
        return y


    def _rebalance(node):
        if node is None:
            return None

        _update_height(node)
        balance = _balance_factor(node)

        if balance > 1:
            if _balance_factor(node.left) < 0:
```

```
            node.left = _rotate_left(node.left)
        return _rotate_right(node)

    if balance < -1:
        if _balance_factor(node.right) > 0:
            node.right = _rotate_right(node.right)
        return _rotate_left(node)

    return node


def avl_insert(root, name, price):
    if root is None:
        return DishNode(name, price)

    if name.lower() < root.name.lower():
        root.left = avl_insert(root.left, name, price)
    elif name.lower() > root.name.lower():
        root.right = avl_insert(root.right, name, price)
    else:
        root.price = price
        return root

    return _rebalance(root)


def avl_search(root, name):
    if root is None:
        return None

    if name.lower() < root.name.lower():
        return avl_search(root.left, name)
    elif name.lower() > root.name.lower():
        return avl_search(root.right, name)
    else:
        return root

def avl_delete(root, name):
    if root is None:
        return None
    if name.lower() < root.name.lower():
        root.left = avl_delete(root.left, name)
    elif name.lower() > root.name.lower():
        root.right = avl_delete(root.right, name)
    else:
        if root.left is None:
            return root.right
        if root.right is None:
            return root.left
        min_right = root.right
        while min_right.left:
```

```
                min_right = min_right.left

            min_right_original_name = min_right.name

            root.name = min_right.name
            root.price = min_right.price

                        root.right  =  avl_delete(root.right,
min_right_original_name)

        _update_height(root)
        return _rebalance(root)

    def avl_size(root):
        if root is None:
            return 0
        return 1 + avl_size(root.left) + avl_size(root.right)


    def avl_height(root):
        return _height(root)

    class TreeIterator:
        def __init__(self, root):
            self.stack = Stack()
            self._push_left(root)

        def _push_left(self, node):
            while node:
                self.stack.push(node)
                node = node.left

        def __iter__(self):
            return self

        def __next__(self):
            if self.stack.is_empty():
                raise StopIteration

            node = self.stack.pop()

            if node.right:
                self._push_left(node.right)

            return (node.name, node.price)


    class MenuManager:
        def __init__(self):
            self.root = None
```

```
    def add(self, name, price):
        if price <= 0:
            return False
        self.root = avl_insert(self.root, name, price)
        return True

    def remove(self, name):
        if self.root is None:
            return False
        old_root = self.root
        self.root = avl_delete(self.root, name)
        return self.root != old_root

    def get_price(self, name):
        node = avl_search(self.root, name)
        return node.price if node else None

    def __iter__(self):
        return TreeIterator(self.root)

    def print_menu(self):
        if self.root is None:
            print("Меню пусто")
            return

        print("\n" + "="*50)
        print("МЕНЮ СТОЛОВОЙ 'МАРИЯ'")
        print("="*50)

        for name, price in self:
            print(f"  {name:35} {price:5d} руб.")

        print("="*50)
        print(f"Всего блюд: {avl_size(self.root)}")
        print(f"Высота дерева: {avl_height(self.root)}")

    def count(self):
        return avl_size(self.root)

    def clear(self):
        self.root = None
```

Файл benchmark.py:

```
import random
import time
import matplotlib.pyplot as plt
import numpy as np
from cw import MenuManager

dish_names = [
```

```
          "Суп харчо", "Куриные котлеты", "Салат греческий", "Суп-лапша", "Шашлык из свинины",
           "Плов узбекский", "Блины с мясом", "Окрошка на квасе", "Котлеты по-домашнему", "Гречка с грибами",
           "Жаркое по-русски", "Картофель по-деревенски", "Рыба в кляре", "Паста карбонара", "Рагу овощное",
           "Лазанья с мясом", "Омлет с сыром", "Солянка сборная", "Салат цезарь", "Куриный бульон",
           "Харчо по-грузински", "Рассольник домашний", "Голубцы с фаршем", "Вареники с картошкой", "Ребра BBQ",
             "Ризотто с грибами", "Картофель запеченный", "Суши ассорти", "Роллы Филадельфия", "Том Ям с креветками",
            "Баклажаны на гриле", "Картофельные драники", "Пицца Пепперони", "Торт Прага", "Кекс маффин",
            "Тирамису классический", "Печенье овсяное", "Маффины черничные", "Фондан шоколадный", "Эклеры заварные",
           "Пирог с вишней", "Морс клюквенный", "Смузи ягодный", "Капучино", "Чай зеленый",
             "Компот из сухофруктов", "Коктейль молочный", "Сок апельсиновый", "Вода газированная", "Молоко топленое"
     ]

     def generate_dish_data(size, case='average'):
         """Генерация данных с реальными названиями блюд"""
         data = []

         if case == 'best':
             for i in range(size):
                 base_name = dish_names[i % len(dish_names)]
                 name = f"{base_name} #{i+1}"
                 price = random.randint(100, 500)
                 data.append((name, price))
             random.shuffle(data)

         elif case == 'worst':
             for i in range(size):
                 letter = chr(ord('A') + (i // 100))
                 num = i % 100
                 name = f"{letter} Блюдо {num:03d}"
                 price = random.randint(100, 500)
                 data.append((name, price))
             data.sort(key=lambda x: x[0])

         else:
             for i in range(size):
                 base_name = random.choice(dish_names)
                 name = f"{base_name} #{i+1}"
                 price = random.randint(100, 500)
                 data.append((name, price))
             random.shuffle(data)
```

```
        return data

    def measure_insert_time(size, case='average'):
        manager = MenuManager()
        data = generate_dish_data(size, case)
        start_time = time.perf_counter()
        for name, price in data:
            manager.add(name, price)
        end_time = time.perf_counter()

        total_time = end_time - start_time
        return total_time / size if size > 0 else 0

    def measure_search_time(size, case='average'):
        manager = MenuManager()
        data = generate_dish_data(size, case)

        for name, price in data:
            manager.add(name, price)

        num_searches = min(100, max(10, size // 10))
        search_names = []
        for _ in range(num_searches):
            if random.random() < 0.8:
                name, _ = random.choice(data)
            else:
                name = f"Несуществующее блюдо
{random.randint(100000, 999999)}"
            search_names.append(name)
        start_time = time.perf_counter()
        for name in search_names:
            manager.get_price(name)
        end_time = time.perf_counter()

        return (end_time - start_time) / num_searches

    def measure_delete_time_fixed(size, case='average',
num_measurements=50):
        total_time = 0
        for _ in range(num_measurements):
            manager = MenuManager()
            data = generate_dish_data(size, case)

            all_names = []
            for name, price in data:
                manager.add(name, price)
                all_names.append(name)
            delete_name = random.choice(all_names)
            start_time = time.perf_counter()
            manager.remove(delete_name)
            end_time = time.perf_counter()
```

```
            total_time += (end_time - start_time)

        return total_time / num_measurements

    def measure_traversal_time(size, case='average'):
        manager = MenuManager()
        data = generate_dish_data(size, case)
        for name, price in data:
            manager.add(name, price)
        start_time = time.perf_counter()
        for _ in manager:
            pass
        end_time = time.perf_counter()

        return end_time - start_time

    def run_benchmark():
        operations = ['insert', 'search', 'delete', 'traversal']
        cases = ['best', 'average', 'worst']
           case_symbols = {'best': 'л', 'average': 'с', 'worst':
'x'}

        sizes = [10, 50, 100, 200, 500, 1000]
        results = {}

        for op in operations:
            results[op] = {}
            for case in cases:
                results[op][case] = []

            print(f"\nИзмерение: {op}")
            for size in sizes:
                print(f"  n={size:4}: ", end="")

                for case in cases:
                    total_time = 0
                    iterations = 5

                    for _ in range(iterations):
                        if op == 'insert':
                              time_val = measure_insert_time(size,
case)
                        elif op == 'search':
                              time_val = measure_search_time(size,
case)
                        elif op == 'delete':
                                                          time_val  =
measure_delete_time_fixed(size, case)
                        elif op == 'traversal':
```

```
                                                                    time_val =
measure_traversal_time(size, case)

                        total_time += time_val

                    avg_time = total_time / iterations
                    results[op][case].append(avg_time)

                    symbol = case_symbols[case]
                    if op == 'traversal':
                        if avg_time < 0.001:
                             print(f"{symbol}:{avg_time*1000:6.2f}
мс ", end="")
                        else:
                              print(f"{symbol}:{avg_time:6.4f}с ",
end="")
                    else:
                        if avg_time < 0.000001:
                              print(f"{symbol}:{avg_time*1e6:6.1f}
µс ", end="")
                        elif avg_time < 0.001:
                             print(f"{symbol}:{avg_time*1000:6.3f}
мс ", end="")
                        else:
                              print(f"{symbol}:{avg_time:7.6f}с ",
end="")

                print()

        return results, sizes

    def create_separate_plots(results, sizes):
        """Создает 4 отдельных графика (по одному на операцию)"""

        operations = ['insert', 'search', 'delete', 'traversal']

        operation_names = {
            'insert': 'Вставка',
            'search': 'Поиск',
            'delete': 'Удаление',
            'traversal': 'Обход'
        }

        case_names = {
            'best': 'Лучший случай',
            'average': 'Средний случай',
            'worst': 'Худший случай'
        }

          colors = {'best': 'green', 'average': 'blue', 'worst':
'red'}
```

```
        markers = {'best': 'o', 'average': 's', 'worst': '^'}

        for op in operations:
            plt.figure(figsize=(10, 7))
            for case in ['best', 'average', 'worst']:
                times = results[op][case]
                     plt.plot(sizes, times, marker=markers[case],
linewidth=2,
                              markersize=8, label=case_names[case],
color=colors[case])

            if op == 'traversal':
                 if len(results[op]['average']) > 2 and sizes[2] >
0:
                       scale_factor = results[op]['average'][2] /
sizes[2]
                        theoretical = [scale_factor * n for n in
sizes]
                             plt.plot(sizes, theoretical, 'k--',
linewidth=2, alpha=0.7,
                             label='Теоретическая O(n)')
            else:
                 if len(results[op]['average']) > 2 and sizes[2] >
0:
                    n_log = [np.log2(max(2, n)) for n in sizes]
                       scale_factor = results[op]['average'][2] /
n_log[2]
                      theoretical = [scale_factor * nl for nl in
n_log]
                             plt.plot(sizes, theoretical, 'k--',
linewidth=2, alpha=0.7,
                             label='Теоретическая O(log n)')

            plt.xlabel('Количество элементов, n', fontsize=12)

            if op == 'traversal':
                           plt.ylabel('Время полного обхода, с',
fontsize=12)
                plt.yscale('linear')
                plt.xscale('linear')
            else:
                      plt.ylabel('Среднее время на операцию, с',
fontsize=12)
                plt.yscale('log')
                plt.xscale('log')

plt.gca().xaxis.set_major_formatter(plt.ScalarFormatter())

plt.gca().xaxis.set_minor_formatter(plt.NullFormatter())
```

```
                plt.title(f'Операция {operation_names[op]} - AVL-дерево', fontsize=14)
            plt.legend(fontsize=10)
                        plt.grid(True, alpha=0.3, which='both', linestyle='--', linewidth=0.5)

            filename = f"avl_{op}.png"
            plt.savefig(filename, dpi=300, bbox_inches='tight')
            plt.close()
    if __name__ == "__main__":
        results, sizes = run_benchmark()
        create_separate_plots(results, sizes)
```

Файл visualization.py:

```
    import random
    import graphviz
    from cw import MenuManager

    dish_names = [
         "Суп харчо", "Куриные котлеты", "Салат греческий", "Суп-лапша", "Шашлык из свинины",
          "Плов узбекский", "Блины с мясом", "Окрошка на квасе", "Котлеты по-домашнему", "Гречка с грибами",
           "Жаркое по-русски", "Картофель по-деревенски", "Рыба в кляре", "Паста карбонара", "Рагу овощное",
          "Лазанья с мясом", "Омлет с сыром", "Солянка сборная", "Салат цезарь", "Куриный бульон",
          "Харчо по-грузински", "Рассольник домашний", "Голубцы с фаршем", "Вареники с картошкой", "Ребра BBQ",
             "Ризотто с грибами", "Картофель запеченный", "Суши ассорти", "Роллы Филадельфия", "Том Ям с креветками",
            "Баклажаны на гриле", "Картофельные драники", "Пицца Пепперони", "Торт Прага", "Кекс маффин",
             "Тирамису классический", "Печенье овсяное", "Маффины черничные", "Фондан шоколадный", "Эклеры заварные",
           "Пирог с вишней", "Морс клюквенный", "Смузи ягодный", "Капучино", "Чай зеленый",
             "Компот из сухофруктов", "Коктейль молочный", "Сок апельсиновый", "Вода газированная", "Молоко топленое"
    ]

    def create_tree_visualization(root):
        graph = graphviz.Digraph(format="png")
            graph.attr('node', shape='circle', fontname='Arial', fontsize='10')

        def add_node(node, parent=None):
            if not node:
                return
            node_label = f"{node.name}\n({node.price}₽)"
```

```
            graph.node(node_label)

            if parent:
                graph.edge(parent, node_label)

            if node.left:
                add_node(node.left, node_label)
            if node.right:
                add_node(node.right, node_label)

        if root:
            add_node(root)

        return graph


    def demonstrate_menu_operations():
              print("ДЕМОНСТРАЦИЯ  AVL-ДЕРЕВА  ДЛЯ  МЕНЮ  СТОЛОВОЙ
'МАРИЯ'")
        menu = MenuManager()
        print("\n1. ДОБАВЛЕНИЕ БЛЮД В МЕНЮ")
        print("-" * 40)

        added_dishes = []
        for _ in range(12):
            name = random.choice(dish_names)
            price = random.randint(50, 1000)

            if menu.get_price(name) is None:
                if menu.add(name, price):
                    added_dishes.append((name, price))
                    print(f"   ✓ {name:25} - {price:4d} руб.")

        print(f"\n   Всего блюд: {menu.count()}")
        print("\n2. ПОИСК БЛЮД В МЕНЮ")
        print("-" * 40)
        if added_dishes:
            for name, price in added_dishes[:3]:
                found_price = menu.get_price(name)
                if found_price:
                        print(f"    ✓ {name:25} - {found_price:4d}
руб.")

             for  test_name  in  ["Суп  харчо",  "Пицца  Пепперони",
"Капучино"]:
            if menu.get_price(test_name) is None:
                print(f"   ✗ {test_name:25} - нет в меню")

        print("\n3. ОТСОРТИРОВАННОЕ МЕНЮ")
        print("-" * 40)
        print("   Название блюда                Цена")
```

```
        print("   " + "-" * 35)

        for name, price in menu:
            print(f"   {name:25} {price:5d} руб.")

        graph = create_tree_visualization(menu.root)
              graph.attr(label='AVL-дерево меню столовой
"Мария"\n(Автоматическая сортировка по названию)',
                  labelloc='t', fontsize='14', fontname='Arial')

        output_file = "avl_tree_visualization"
        graph.render(output_file, cleanup=True)

        print("\n4. УДАЛЕНИЕ БЛЮДА ИЗ МЕНЮ")
        print("-" * 40)

        if added_dishes:
                      dish_to_remove_name, dish_to_remove_price =
added_dishes[0]
               exists_before = menu.get_price(dish_to_remove_name)
is not None
            menu.remove(dish_to_remove_name)
             exists_after = menu.get_price(dish_to_remove_name) is
not None
            if exists_before and not exists_after:
                     print(f"    ✓ Удалено: {dish_to_remove_name}
({dish_to_remove_price} руб.)")
                print(f"   ✓ Осталось блюд: {menu.count()}")
                print("\n   Оставшиеся блюда в меню:")
                remaining_dishes = []
                for name, price in menu:
                    remaining_dishes.append((name, price))

                                  for i, (name, price) in
enumerate(remaining_dishes[:10], 1):
                        print(f"   {i:2}. {name:25} - {price:4d}
руб.")

                if len(remaining_dishes) > 10:
                     print(f"   ... и ещё {len(remaining_dishes) -
10} блюд")
            else:
                if not exists_before:
                      print(f"   ✗ Блюдо {dish_to_remove_name} не
найдено в меню")
                else:
                       print(f"   Блюдо {dish_to_remove_name} все
еще в меню (ошибка удаления)")

     if __name__ == "__main__":
        demonstrate_menu_operations()
```

# ПРИЛОЖЕНИЕ Б
# ТЕСТИРОВАНИЕ

| № п/п | Входные данные | Ожидаемый результат | Комментарии |
|---|---|---|---|
| 1. | Создание пустого стека | is_empty() == True, size == 0 | Корректная инициализация |
| 2. | push("тест"), pop() | pop() == "тест", стек пуст | Базовая работа LIFO |
| 3. | push("первый"), push("второй") | pop() == "второй", pop() == "первый" | Соблюдение порядка LIFO |
| 4. | push("элемент"), peek() | peek() == "элемент", size == 1 | Peek не изменяет стек |
| 5. | insert("Борщ", 150), insert("Суп", 120), search("Борщ") | price == 150, search("Нет") == None | Корректность вставки и поиска |
| 6. | insert("Борщ", 150), insert("Борщ", 180) | price == 180, size == 1 | Обновление существующего элемента |
| 7. | insert("Борщ", 150), insert("Суп", 120), delete("Суп") | search("Суп") == None, size == 1 | Удаление элемента |
| 8. | Вставка 3 элементов, удаление корня | Дерево сохраняет структуру, size == 2 | Удаление корня с балансировкой |
| 9. | Последовательная вставка 5 элементов | Высота дерева ≤ 3 | Балансировка малого дерева |

| | | | |
|---|---|---|---|
| 10. | Вставка 100 элементов | Высота ≤ $1.44 \times \log_2(102)$ | Гарантированная высота АВЛ-дерева |
| 11. | Вставка в порядке "Десерт", "Борщ", "Суп" | Итератор возвращает ["Борщ", "Десерт", "Суп"] | Лексикографический порядок |
| 12. | Создание итератора для None | Возвращает пустой список | Обработка пустого дерева |
| 13. | Вставка 10 элементов, обход итератором | Возвращает 10 элементов | Полнота обхода |
| 14. | Создание пустого меню | count() == 0, get_price() == None | Корректная инициализация |
| 15. | add("Борщ", 150), add("Суп", 120) | count() == 2 | Добавление валидных блюд |
| 16. | add("Паста", 0), add("Пицца", -100) | Возвращает False, count () == 0 | Валидация цены (>0) |
| 17. | add("Борщ", 150), add("Борщ", 180), get_price("Борщ") | Цена обновлена на 180 | Обновление существующего блюда |
| 18. | add("Борщ", 150), remove("Борщ"), remove("Нет") | Успешное удаление, при несуществующем False | Удаление с проверкой |
| 19. | add("борщ", 150), add("БОРЩ", 200) | count() == 1, цена 200 | Регистронезависимость операций |
| 20. | Полный сценарий: добавление, обновление, удаление | Все операции корректны, порядок сохранен | Проверка соответствия условиям задачи |